

Production Deployment, DevOps & Infrastructure Runbook

Containerization, Reverse Proxy Configuration, Process Supervision & High-Availability Operations

Academic and strategic research artifact for the AI-Powered Household Appliance Rental System (RentAI / SDC2). Demonstrating circular economy transition, machine learning churn mitigation, FunkSVD recommendation, and localized generative AI edge deployment.

Document Ref:	DEP-SDC2-2026-V2.4	Publication:	September 2026
Project Title:	RentAI / SDC2 Autonomous Rental Platform	Peer Review:	Academic Capstone & IEEE/ACM Format
Principal Authors:	Advanced AI & Systems Research Group	Affiliation:	Department of Computer Science & Engineering
Classification:	Scholarly Research & Field Engineering	Status:	PUBLISHED & ARCHIVED

1. Production Topology & Architecture

The production deployment topology isolates web traffic, API routing, database persistence, and local AI computation across coordinated system services.

```
[Internet / Clients]
|
v  HTTPS (Port 443)
[Nginx Reverse Proxy & SSL Offloader]
|
+----> /api/*  -----> [Gunicorn / Waitress WSGI Gateway (Port 8000)]
|
|                                     +----> MongoEngine -----> [MongoDB 8.3 (Port 27017)]
|                                     |
|                                     +----> HTTP/REST -----> [Ollama AI Daemon (Port 11434)]
|
+----> /*      -----> [Vite Static SPA HTML/JS/CSS Chunks]
```

2. Containerization: Docker & Docker-Compose

The entire platform is deployable via multi-stage container builds:

```
# docker-compose.prod.yml
version: '3.8'
services:
  mongodb:
    image: mongo:8.3
    restart: always
    volumes:
      - mongo_data:/data/db
    ports:
      - "27017:27017"

  backend:
    build: ./backend
    restart: always
    command: gunicorn core.wsgi:application --bind 0.0.0.0:8000 --workers 4
    depends_on:
      - mongodb
    ports:
      - "8000:8000"

  frontend:
    build: ./frontend
    restart: always
    ports:
      - "80:80"
```

3. Process Supervision & Self-Healing Daemons

Under Windows, background services are managed via PowerShell jobs or NSSM service wrappers. Under Linux, `systemd` unit files guarantee automatic service restart upon process interruption or system reboot:

```
[Unit]
Description=RentAI Django Application Gateway
After=network.target mongodb.service

[Service]
User=rentai
WorkingDirectory=/opt/rentai/backend
ExecStart=/opt/rentai/venv/bin/gunicorn core.wsgi:application --bind 127.0.0.1:8000 --workers 4
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target
```

4. Backup & Disaster Recovery Protocols

MongoDB catalog and transactional states are preserved through automated nightly snapshot dumps:

```
mongodump --db appliance_rental --out /var/backups/mongodb/$(date +%F)
```